Project Invincible: Updated System Architecture

1. System Architectural Philosophy and Core Framework

Project Invincible is engineered as a high-performance, native **Multithreaded C++ Core** . This architectural decision is a direct rejection of the modern trend toward web-based frameworks like Electron. For a stealth-focused application, Electron is a liability; its Chromium overhead creates a massive memory footprint, high CPU idle states, and—most critically—a predictable process signature that is trivial for anti-cheat and monitoring software to flag.By utilizing C++20, we achieve a degree of low-level OS control that ensures total invisibility. The system operates "near the metal," allowing us to hook into Win32 APIs and manipulate memory buffers directly. This provides a significant competitive moat: while competitors struggle with the "bloat" of JavaScript bridges, Invincible delivers sub-millisecond execution. From an investor-readiness perspective, this translates to a tiny binary size (<20MB), zero dependency on local runtimes, and significantly lower distribution and scaling costs. We have balanced this low-level performance with a modular AI interface, ensuring that the "IntelligenceCore" remains decoupled from the high-speed data acquisition layer.

2. Subsystem Deep Dive: Stealth and Visualisation (GhostUI)

The "GhostUI" is the system's primary output mechanism, designed to render an "Invincible State" HUD that is physically invisible to screen-recording software.Rather than using standard GDI or GDI+ which are easily captured by the Desktop Duplication API, GhostUI utilizes a hardware-accelerated **Direct2D** swap chain. By leveraging the GPU for rendering, we achieve zero-latency alpha blending and superior text clarity through sub-pixel antialiasing. Stealth is enforced by manipulating the Win32 SetWindowDisplayAffinity API, setting the flag to WDA_EXCLUDEFROMCAPTURE. This ensures that while the user sees a high-fidelity HUD, the OS-level capture stream receives a null buffer for the window's coordinates.**GhostUI Technical Specifications:**

- **Rendering Pipeline:** Direct2D hardware-accelerated swap chain.
- **Refresh Target:** 60 FPS locked for fluid overlay integration.
- **Stealth Mechanism:** SetWindowDisplayAffinity (WDA_EXCLUDEFROMCAPTURE) enforcement.
- **Visual Fidelity:** ClearType-compatible sub-pixel antialiasing with per-pixel alpha transparency.
- **Memory Footprint:** <15MB VRAM allocation.

3. Subsystem Deep Dive: Data Acquisition (AudioSentinel & VisionAware)

The acquisition layer is designed to ingest environmental data without triggering modern Windows "Privacy Indicators" (such as the green microphone icon or yellow screen-capture border).**AudioSentinel** This module utilizes **WASAPI (Windows Audio Session API) Loopback** to intercept the PCM output buffers from the sound card. By operating at the session level, we capture meeting audio without needing to join the call as a participant. The module implements a dual-channel capture logic:

1. **Device-Out:** Intercepts the "Speaker" (interviewer) stream.
2. **Device-In:** Intercepts the "Mic" (user) stream via a non-exclusive capture hook. This separation allows the IntelligenceCore to differentiate between external queries and

internal responses, providing critical conversational context.**VisionAware (Screen Capture Module)** VisionAware is a conceptual conceptual bypass of the standard Windows.Graphics.Capture API. To avoid the tell-tale "yellow border" notification in Windows 10/11, VisionAware utilizes low-level GDI bit-block transfers (BitBlt) or the Desktop Duplication API (DXGI) on a specific frame-polling interval. This allows for stealthy "display awareness," scraping application metadata and screen context (like a coding prompt or a technical diagram) while remaining invisible to the host application's anti-tamper checks.**Data Acquisition Layer Summary**| Module | Technology | Primary Function || ------ | ------ | ------ || AudioSentinel | WASAPI Loopback | Intercepts PCM output/input buffers for speaker separation. || VisionAware | DXGI/GDI BitBlt | Stealthy screen-context acquisition bypassing capture notifications. || SyncManager | Producer-Consumer Queue | Synchronizes audio/visual timestamps for context-aware AI. |

4. Subsystem Deep Dive: Processing and Intelligence (StreamProcessor & IntelligenceCore)

StreamProcessor To eliminate the latency and privacy risks of streaming raw audio to the cloud, the StreamProcessor utilizes a local **Whisper.cpp** implementation. The audio pipeline is managed via a dedicated worker thread to ensure the main UI loop never blocks.**Sliding Window & VAD Logic:**

1. **VAD Trigger:** A local Voice Activity Detection (VAD) filter monitors the WASAPI buffer, identifying speech vs. ambient noise.
2. **Window Initialization:** Upon VAD detection, a "Sliding Window" begins accumulating audio frames in a local ring buffer.
3. **Context Preservation:** When silence is detected, the window finalizes, but preserves the trailing 500ms of the previous window to ensure phonetic continuity and prevent "clipped" starts.
4. **Transcription:** The finalized segment is processed by the Whisper.cpp tiny.en or base.en model, returning text in <200ms.**IntelligenceCore** The "Brain" of the system utilizes **Gemini 1.5 Flash** for its massive context window and low inference cost. Through "System Prompting," the AI is instructed to act as a stealth "co-founder." It doesn't just transcribe; it synthesizes. It continuously ranks the importance of incoming transcripts against the "ContextStore," ensuring that the synthesized output is grounded in the user's actual history.

5. Subsystem Deep Dive: Persistence and Memory (ContextStore)

The **ContextStore** utilizes a local **SQLite** database augmented with a **Vector Search** extension (such as sqlite-vss). This enables Retrieval-Augmented Generation (RAG) during live sessions.

- **Ingestion:** Users drop PDFs or markdown files into the app. The system parses these via a background worker, chunks the text, and generates embeddings.
- **Contextual Prioritization:** During a live interview, the system doesn't just rely on the transcript. It performs a cosine similarity search between the last 30 seconds of conversation and the stored embeddings.
- **Result Injection:** Relevant excerpts from the user's resume or project history are prioritized and injected into the IntelligenceCore prompt, allowing the AI to say, "Based on my work on Project X..." rather than providing generic answers.

6. The "Perfect" System Flow: End-to-End Execution

1. **Background Launch:** The system initializes the C++ core and GhostUI (hidden).
2. **Indexing:** The user "feeds" the ContextStore with their technical background.
3. **Stealth Capture:** Upon meeting start, AudioSentinel begins pulling PCM data via WASAPI. VisionAware polls the screen at a low frequency to identify technical keywords.
4. **Local Inference:** Whisper.cpp transcribes speech segments locally on a background thread using the sliding window algorithm.
5. **Intelligence Synthesis:** The text is sent to Gemini 1.5 Flash. The IntelligenceCore queries the ContextStore for vector matches, combining the "Who you are" (stored) with the "What they asked" (live).
6. **HUD Delivery:** The AI response is rendered to the user via the Direct2D GhostUI. Because of the SetWindowDisplayAffinity flag, the response is invisible to the interviewer, even if the user is sharing their entire screen.

7. Professional Directory Structure

The project employs a modular PIMPL (Pointer to Implementation) pattern where possible to hide implementation details and speed up compile times./ProjectInvincible /assets /fonts (Custom monospaced HUD fonts) /icons (System tray stealth icons) /include /api (Public headers for internal modules) /common (Shared types and logging macros) /lib /whisper (Local Whisper.cpp binaries) /sqlite (SQLite + Vector search extensions) /src /audio (AudioSentinel: WASAPI implementation) /core (Main entry, WinMain, and Thread Pool) /gui (GhostUI: Direct2D renderer and Window manager) /intelligence (IntelligenceCore: Gemini API and Prompt Logic) /storage (ContextStore: SQLite-vss and File Parser) /vision (VisionAware: DXGI/GDI capture hooks) /tests /unit (Module-level testing) /integration (End-to-end pipeline validation)

8. Technical Constraints and Implementation Directives

- **Zero-Overhead Requirement:** No Node.js, Electron, or Python runtimes. The application must be a standalone Win32 PE (Portable Executable).
- **Performance Concurrency:** The system must utilize a strict Producer-Consumer multithreading model. Audio capture, Whisper transcription, and UI rendering must exist on independent hardware threads.
- **Rendering Integrity:** All overlays must be Direct2D-based to support hardware-level Display Affinity and 60 FPS updates.
- **Latency Ceiling:** The end-to-end pipeline (from audio capture to HUD display) must not exceed a 2.5-second total delay to remain viable for real-time interaction.
- **Network Security:** All external AI calls must be encrypted via TLS 1.3, with optimized payloads to minimize packet inspection signatures.